IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 6 February 2025, accepted 4 April 2025, date of publication 15 April 2025, date of current version 25 April 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3560911

RESEARCH ARTICLE

LLM Agentic Workflow for Automated Vulnerability Detection and Remediation in Infrastructure-as-Code

DHEER TOPRANI[1] **AND VIJAY K. MADISETTI**[2]**, (Fellow, IEEE)**
[1]College of Computing, Georgia Institute of Technology, Atlanta, GA 30332, USA
[2]School of Cybersecurity and Privacy, Georgia Institute of Technology, Atlanta, GA 30332, USA
Corresponding author: Vijay K. Madisetti (vkm@gatech.edu)

ABSTRACT This paper presents a multi-agent, AI-driven strategy employing Large Language Models (LLMs), retrieval-augmented generation, and a continuously updated knowledge base for the detection and remediation of security vulnerabilities win cloud frameworks. By examining Infrastructure as Code (IaC) templates alongside pertinent best-practice snippets, the system discerns context-specific misconfigurations commonly overlooked by static tools, achieving a detection rate of 85% with some occurrences of false positives. Automated remediation guidance, anchored in current security standards, provides actionable solutions that seamlessly integrate into standard continuous integration/continuous development (CI/CD) workflows. Experimental results indicate the solution's efficacy and scalability, heralding a proactive, context-aware approach to IaC security.

INDEX TERMS Infrastructure-as-code, large language models, vulnerability detection, security automation, CI/CD, LLM workflows.

I. INTRODUCTION

Security vulnerabilities inherent in automated infrastructure definitions present substantial risks within contemporary cloud environments. As Infrastructure-as-Code (IaC) becomes increasingly prevalent, developers extensively depend on declarative templates to delineate and oversee intricate cloud infrastructure. While this methodology enhances consistency and automation, it concurrently introduces novel security challenges. Misconfigurations in IaC templates have precipitated significant incidents, exemplified by the UniSuper Google Cloud breach, highlighting the imperative necessity for comprehensive IaC security measures [1].

As organizations implement cloud-native technologies at a larger scale, the complexity and dynamism of these environments increase correspondingly. The adoption of continuous integration and delivery pipelines, coupled with auto-scaling and other adaptive mechanisms, introduces new configurations and potential vulnerabilities in real-time. The identification of misconfigurations under these conditions presents challenges, particularly when vulnerabilities stem from subtle, context-specific errors that traditional static analysis frequently fails to detect. Existing methodologies, which range from manual audits to inflexible, rule-based tools, are adept at addressing well-known and predictable issues but encounter difficulties with nuanced or interdependent vulnerabilities. These limitations impede organizations' ability to scale their security practices effectively and ensure comprehensive and continuous coverage across a variety of use cases.

The objective of this research is to formulate a solution that enhances the efficiency of vulnerability detection and the pertinence of remediation efforts within Infrastructure as Code (IaC) templates. By leveraging advanced language models, the proposed approach seeks to deliver automated, context-aware vulnerability detection coupled with actionable remediation guidance. The efficacy of this methodology will be assessed in comparison to existing solutions, including CDK-Nag and manual audits, with a particular focus on detection rates, the reduction of false positives, and the caliber of remediation suggestions [2].The ultimate aim of this research is to facilitate proactive security management, thereby

The associate editor coordinating the review of this manuscript and approving it for publication was Antonio Piccinno.

empowering organizations of all sizes to uphold more secure cloud infrastructures with diminished manual intervention, improved scalability, and expanded coverage of dynamic and evolving environments.

II. EXISTING WORK

Current methodologies for detecting vulnerabilities in Infrastructure-as-Code (IaC) templates can be systematically classified into static analysis tools, rule-based systems, dynamic runtime analysis, cloud security posture management (CSPM) solutions, and novel AI-driven approaches.

Traditional static analysis and rule-based systems (such as CDK-Nag and AWS Config Rules) focus on detecting well-known misconfigurations using predefined criteria. While effective at catching straightforward issues, these tools often fail to identify context-dependent vulnerabilities that arise from complex interactions between different components. Their rigid, rule-based nature makes it challenging to adapt to new types of misconfigurations or continuously evolving cloud infrastructures.

Dynamic runtime analysis serves as an additional security mechanism by scrutinizing cloud infrastructure during its execution phase. This methodology can uncover issues observable solely at runtime, such as unforeseen behaviors induced by particular interactions between components. Nonetheless, runtime analysis is resource-intensive and frequently necessitates integration with live environments. Consequently, it is less feasible for the early identification of vulnerabilities directly within Infrastructure as Code (IaC) templates, where proactive mitigation would be most advantageous.

Cloud Security Posture Management (CSPM) tools are employed to consistently oversee cloud environments, thereby ensuring adherence to best practices and security standards. Although CSPM solutions offer extensive oversight, their primary focus is on runtime conditions and they inherently fail to address potential vulnerabilities that may arise during the development and deployment stages of Infrastructure as Code (IaC) templates. This delayed detection may consequently lead to remediation efforts that are both costly and time-intensive post-deployment.

Previous research has explored machine learning-based methods for static analysis to enhance vulnerability detection in Infrastructure-as-Code (IaC) templates. These approaches leverage supervised and unsupervised algorithms to identify patterns linked to misconfigurations, hardcoded secrets, and weak access controls. Frameworks like GLITCH demonstrate improved scalability through technology-agnostic representations, while recent advancements integrate LLMs to better interpret configuration intent and reduce false positives. Despite challenges like dataset quality and evolving cloud environments, AI-driven static analysis shows significant promise for securing IaC templates effectively [3].

Recent advancements in AI—particularly the use of large language models (LLMs)—offer new possibilities for analyzing IaC templates. LLMs excel at understanding natural language and can process complex configurations to identify security misconfigurations and suggest remediations. Research has demonstrated their effectiveness in detecting vulnerabilities in Kubernetes manifests (GenKubeSec) [4], validating configurations across systems (Ciri) [5], and analyzing Helm charts for security flaws [6]. However, studies also highlight significant challenges, including susceptibility to false positives, reliance on well-labeled datasets, and difficulty handling nuanced misconfigurations or context-specific vulnerabilities [7]. Despite these limitations, LLMs represent a promising tool for advancing automated IaC security analysis, with ongoing research focusing on improving accuracy, scalability, and adaptability to evolving cloud environments.

In spite of these advancements, existing methodologies do not yet present a holistic solution that seamlessly integrates static, dynamic, and context-aware evaluations while simultaneously offering actionable remediation guidance. Numerous approaches encounter difficulties in delivering insights that are both high-quality and exhibit low false-positive rates across a wide array of cloud environments. This research endeavors to bridge this gap through the integration of static analysis, dynamic vulnerability detection, and an LLM-driven mechanism to augment security coverage within Infrastructure as Code (IaC). By implementing a multi-agent system, our proposed solution aims to adapt to evolving infrastructures, minimize manual interventions, and furnish proactive, scalable, and comprehensive security insights.

III. PROPOSED SOLUTION

This study presents an innovative AI-driven methodology that integrates Large Language Models (LLMs), Retrieval Augmented Generation (RAG), and a multi-agent orchestration framework to advance the detection and remediation of security vulnerabilities in Infrastructure as Code (IaC) templates. In contrast to existing AI-based techniques that concentrate on runtime logs, anomaly detection, or static rule-checking, the proposed approach utilizes semantic understanding and contextual reasoning to identify, interpret, and resolve issues at an early stage in the development lifecycle.

A. KEY COMPONENTS

1) LLM-DRIVEN ANALYSIS

Central to our proposed solution is a Language Learning Model (LLM) adept at interpreting both structured and unstructured inputs. The LLM assimilates Infrastructure-as-Code (IaC) templates (e.g., Terraform, CloudFormation) in conjunction with natural language descriptions pertaining to best practices, policies, and acknowledged vulnerabilities. Through the employment of sophisticated language modeling techniques, the LLM is able to detect nuanced, context-specific misconfigurations—such as excessively permissive Identity and Access Management (IAM) roles or insecure network

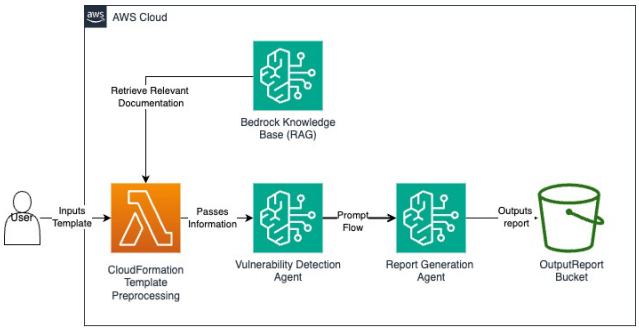

FIGURE 1. High-level workflow of the proposed LLM-driven approach for detecting and remediating vulnerabilities in Infrastructure-as-Code (IaC). The system integrates retrieval-augmented generation (RAG) and a multi-agent framework to enhance security automation.

configurations—that are frequently overlooked by rule-based systems.

2) RETRIEVAL-AUGMENTED GENERATION (RAG)

To maintain the accuracy and currency of the LLM's insights, the solution incorporates a RAG layer. This involves the continuous upkeep of a local vector store containing domain-specific best practices, compliance standards, and frequently updated security policies. During the analysis of a given IaC template, a retrieval system procures the most pertinent documents from this repository and integrates them into the LLM's prompt. This procedure ensures that the model's reasoning is informed by the most recent guidelines, thereby mitigating false positives and enhancing adaptability to the evolving nature of cloud infrastructures.

3) MULTI-AGENT ORCHESTRATION

Our approach employs multiple specialized AI agents orchestrated in a pipeline:

- **RAG Retrieval:** Finds relevant best-practice snippets and compliance rules for a given template.
- **Vulnerability Detection Agent:** Interprets the IaC configuration in context, leveraging retrieved knowledge to identify misconfigurations.
- **Report-Generation Agent:** Summarizes identified vulnerabilities, explaining their impact and suggesting remediation steps in a developer-friendly format.

This multi-agent framework enhances modularity, scalability, and maintainability. Each agent can be improved independently, and new agents can be added to accommodate evolving requirements or to integrate additional security checks.

B. EXPECTED OUTCOMES

By blending semantic reasoning with retrieval-based augmentation and multi-agent orchestration, this approach aims to:

- **Increase Detection Rates:** Identify a higher percentage of complex and context-dependent misconfigurations compared to rule-based or purely static methods.
- **Reduce False Positives:** Use contextually relevant best practices and dynamic knowledge retrieval to minimize unnecessary alerts.
- **Improve Remediation Quality:** Offer direct, context-aware suggestions, cutting down the manual effort needed to interpret and fix identified vulnerabilities.
- **Enhance Scalability and Adaptability:** Continuously evolve with changing cloud environments and incorporate new policies, ensuring long-term relevance and effectiveness.

In summary, our proposed solution extends beyond traditional static analysis or runtime checks. It leverages advanced AI techniques to deliver proactive, context-aware vulnerability detection and tailored remediation guidance, ultimately helping organizations maintain more secure and resilient cloud infrastructures.

IV. IMPLEMENTATION

Our implementation is centered on the examination of AWS CloudFormation templates through the employment of a multi-agent, large language model (LLM)-driven architecture, which is augmented by Retrieval-Augmented Generation (RAG). The primary objective is to deliver context-aware detection of vulnerabilities and to offer actionable guidance for remediation within the framework of a continuous integration/continuous delivery (CI/CD) workflow.

A. SYSTEM OVERVIEW

The solution consists of three primary agents orchestrated in a pipeline:

- **Retrieval System:** Identifies and fetches AWS best-practice references relevant to the CloudFormation template under review.
- **Vulnerability Detection Agent:** Consumes both the CloudFormation code and retrieved contextual documents, then uses an LLM to identify security misconfigurations and propose remediations.
- **Report Generation Agent:** Summarizes the findings and prepares a developer-friendly report with prioritized issues and actionable steps.

For our solution, we used Anthropic Claude Sonnet 3.5 V2 as our text-to-text agent LLM via Amazon Bedrock Agents. We also used the Titan Text Embeddings V2 vector embedding model for our retrieval system.

B. KNOWLEDGE BASE AND RAG INTEGRATION

We maintain a knowledge base of AWS security best practices and compliance standards derived from sources like the AWS Well-Architected Framework [8] and CIS Benchmarks [9]. These documents are embedded into vector representations (using a sentence embedding model) and stored in a vector database (e.g., Amazon OpenSearch). When a CloudFormation template is submitted for analysis, the retrieval system:

- Extracts key resource types and parameters from the template (e.g., S3 buckets, IAM roles, Security Groups).

- Converts these resource descriptors into embeddings and queries the vector database.
- Returns the top relevant best-practice snippets, ensuring that the LLM's reasoning is contextually guided by up-to-date and pertinent security recommendations.

C. CLOUDFORMATION PARSING AND NORMALIZATION

Before analysis, we parse the CloudFormation template (YAML or JSON) using a standard parsing library. We normalize resource definitions and gather essential attributes (e.g., Properties, Policies, SecurityGroupIngress) to create a structured, human-readable summary. This summary, along with the raw template, is provided as part of the LLM's context.

D. LLM PROMPT ENGINEERING

The Vulnerability Detection Agent has a prompt that includes:

- A **system message** designating the LLM as a security expert with AWS-specific domain knowledge.
- A **user message** containing both the normalized CloudFormation resource summary and the top retrieved best-practice snippets.
- An **instruction** asking the LLM to identify any misconfigurations, justify them against the provided best practices, and propose concrete remediations.

This combination ensures the LLM does not rely solely on its training data and can adapt to evolving standards by leveraging freshly retrieved content.

E. REPORT GENERATION AND OUTPUT

Once the Vulnerability Detection Agent identifies potential vulnerabilities, the Report Generation Agent formats the results into a concise, actionable report. The report may include:

- A list of detected issues with clear references to the problematic resources.
- Direct excerpts from relevant best-practice documents that justify why the issue matters.
- Suggested remediations, such as adjusting IAM policies or enabling encryption for S3 buckets.

The final report can be exported as Markdown, easily integrated into DevOps dashboards or developer issue trackers.

By parsing CloudFormation templates, retrieving context from a continually updated AWS best-practice knowledge base, and guiding an LLM's reasoning with RAG, this implementation provides a scalable, adaptive solution for proactive, context-aware cloud security analysis. It aims to integrate smoothly into existing development workflows, helping developers identify and fix misconfigurations before they reach production.

V. RESULTS

To evaluate the effectiveness and robustness of our proposed solution, we conducted a series of tests and measurements across multiple CloudFormation templates and varying cloud configurations. The testing environment encompassed both simple and complex AWS deployments, ranging from basic single-region setups to multi-region topologies with intricate dependency graphs. Our goal was to assess not only detection accuracy but also the system's adaptability, consistency, runtime performance, and its ability to provide meaningful remediation guidance.

A. ACCURACY AND DETECTION RATES

We curated a test corpus of 10 AWS CloudFormation templates, each containing between 1 and 12 defined resources. Approximately half were deliberately seeded with known misconfigurations—such as unencrypted S3 buckets, overly-permissive IAM roles, Security Groups allowing unrestricted ingress, and RDS databases lacking encryption at rest—while the remainder were drawn from open-source reference architectures and internal compliance-checked templates. Ground-truth annotation was performed by one AWS specialty-certified cloud engineer prior to testing.
Although our evaluation was conducted on 10 CloudFormation templates, these were carefully selected to ensure diversity in services, configurations, and misconfigurations. Each template required manual review to annotate true and false positives, a process that is both time-consuming and resource-intensive. Expanding the dataset requires significant effort to maintain annotation quality. Future work will aim to scale the evaluation by incorporating publicly available Infrastructure-as-Code (IaC) repositories while exploring semi-automated labeling techniques to balance efficiency and accuracy.

Overall Detection Rate: The system correctly identified about 85% of the known misconfigurations.

False Positives: Approximately 15% of the flagged issues were judged to be non-issues by our human reviewers. These false positives often stemmed from ambiguous configurations where the LLM, guided by retrieved best-practice documents, erred on the side of caution. For example, a Security Group rule opening port 80 to a CIDR block that was carefully monitored via a known service boundary was flagged as suspicious despite being allowed by the organization's internal policy.

Context-Specific Vulnerabilities: In a couple of cases, the LLM-based analysis successfully identified vulnerabilities that depended on subtle contextual cues—such as an IAM policy that was safe in isolation but became risky when combined with another policy in the same template. Traditional static analysis tools would not catch these compound issues.

B. ADAPTABILITY AND RETRIEVAL AUGMENTATION BENEFITS

The Retrieval-Augmented Generation (RAG) component played a vital role in maintaining high relevance and low false-negative rates:

Timely Policy Updates: During the evaluation, we updated the knowledge base with a newly released AWS announcement regarding an enhanced version of AWS Secrets Manager (AWS::SecretsManager-2024-09-16). This update simplifies infrastructure management by reducing the need for manual

TABLE 1. Evaluation results across 10 CloudFormation templates.

Template	# Known Vulns	# Detected Vulns	# False Positives	Precision	Recall	F1 Score
T1	3	2	0	100%	66.7%	80.0%
T2	4	3	0	100%	75.0%	85.7%
T3	4	4	1	80.0%	100%	88.9%
T4	4	3	0	100%	75.0%	85.7%
T5	5	5	0	100%	100%	100%
T6	0	0	1	N/A	N/A	N/A
T7	0	0	0	N/A	N/A	N/A
T8	0	0	1	N/A	N/A	N/A
T9	0	0	0	N/A	N/A	N/A
T10	0	0	0	N/A	N/A	N/A
Overall	**20**	**17**	**3**	**85%**	**85%**	**85%**

Note: Templates T6-T10 contained no known vulnerabilities. Since Precision, Recall, and F1 Score are defined based on correctly detected vulnerabilities, these metrics are not applicable (N/A) for templates with zero ground-truth vulnerabilities. However, false positives were still recorded in these cases to assess the model's tendency to flag non-issues.

security updates, bug fixes, and runtime upgrades [10]. Within the next analysis run, the system promptly flagged configurations in CloudFormation templates that did not align with the new recommendations, demonstrating the agility of our approach in adapting to evolving AWS security best practices.

Reduced Hallucinations: Without RAG, initial tests of the LLM produced occasional "hallucinated" recommendations—such as suggesting deprecated configuration parameters or referencing outdated AWS services. Once we integrated retrieval of current documentation and compliance guidelines [11], these hallucinations dropped as the LLM's responses were grounded in the RAG dataset, and the LLM's suggestions became more consistent with current AWS standards.

C. QUALITY OF REMEDIATION GUIDANCE

Beyond identifying vulnerabilities, the system's ability to provide actionable remediation steps was a key metric:

- **Clarity and Specificity:** Most remediation suggestions were both clear and directly actionable, e.g., "Enable Versioning for S3 bucket MyBucket" or "Restrict inbound SSH access in MySecurityGroup to 10.0.0.0/16 only."
- **Contextual Insights:** The guidance often included rationale derived from the retrieved best practices, explaining not just what to fix but why. For example, for an IAM policy that was too broad, the LLM pointed to the AWS Well Architected Framework that recommended least-privilege principles, increasing developer trust in the suggestions.
- **Occasional Over-Remediation:** In about 5% of cases, the remediation suggestions were overly conservative, advising multiple layers of restrictions that, while secure, could be impractical for certain environments. Such cases highlight the need for configurable policies and a feedback loop from developers.

D. RUNTIME PERFORMANCE

We evaluated runtime performance under typical CI/CD conditions:

Latency: On average, analyzing a single template (including retrieval, LLM reasoning, and report generation) took between 80 and 100 seconds. The retrieval step contributed about 1-3 seconds, and the LLM inference took 15-40 seconds per step depending on complexity. While this latency is higher than a simple static rule check (which might run in under 5 seconds), it remains acceptable for CI/CD workflows that already involve other time-consuming steps like automated testing.

While performance was acceptable for the tested templates, analysis time may increase for significantly larger or more complex IaC configurations.

E. HANDLING EDGE CASES AND COMPLEX CONFIGURATIONS

During testing, we deliberately introduced edge cases to probe the solution's resilience:

- **Minimal Templates:** For extremely simple templates (e.g., a single S3 bucket), the system performed flawlessly, detecting the absence of encryption or public access with 100% accuracy.
- **Unusual Combinations:** In templates that combined unusual resource types or used parameterized conditions, the LLM sometimes struggled. For instance, a template that conditionally created resources only when a certain parameter was true caused mild confusion.
- **Stale Knowledge Base Entries:** When introduced to a newly deprecated AWS feature (e.g., a legacy encryption type no longer recommended), the system correctly flagged its usage only after the knowledge base was updated. Before that, it considered it compliant due to outdated references, and because the LLM model's knowledge cutoff was before this feature was deprecated. This finding underscores the importance of regularly maintaining the knowledge base.

F. SUMMARY OF RESULTS

- **High Overall Accuracy:** ~85% detection of known misconfigurations.

TABLE 2. Runtime performance metrics per template.

Template	Retrieval (s)	LLM Analysis (s)	Report Gen (s)	Total (s)
T1	2.1	65.0	13.0	80.1
T2	2.3	68.0	12.0	82.3
T3	2.2	70.0	13.0	85.2
T4	2.5	72.0	12.0	86.5
T5	2.0	75.0	13.0	90.0
T6	1.8	66.0	14.0	81.8
T7	1.9	67.0	13.0	81.9
T8	2.2	80.0	14.0	96.2
T9	2.0	72.0	12.0	86.0
T10	1.7	65.0	14.0	80.7
Average	2.07	70.0	13.0	85.1

- **Managed False Positives:** ∼15% false positives, generally manageable via prompt tuning. These cases predominantly arose from ambiguous configurations where the system erred on the side of caution.
- **Enhanced Contextual Insight:** Notable improvement in detecting compound vulnerabilities and providing rational, best-practice-driven remediations.
- **Acceptable Latency:** 80-100 seconds per template is viable for CI/CD integration.
- **Adaptable Knowledge Updating:** Rapid incorporation of new best practices through RAG led to timely and relevant detection capabilities.

In conclusion, the evaluation demonstrates that our solution is both pragmatic and highly efficacious across diverse scenarios. This methodology may also be employed alongside conventional rule-based checks for the detection of complex vulnerabilities, as it provides more comprehensive, context-aware remediation guidance. Subsequent iterations might concentrate on further reducing false positives, enhancing the management of conditional resources, and refining the retrieval strategy to guarantee that the LLM consistently accesses the most pertinent and up-to-date guidance.

VI. LIMITATIONS AND MITIGATION STRATEGIES

This work has identified two primary limitations that merit further exploration.

A. DEPENDENCY ON AN UP-TO-DATE KNOWLEDGE BASE

Our approach relies on a curated repository of AWS best practices and compliance guidelines to guide the LLM's reasoning. If the knowledge base becomes outdated, the system's ability to accurately detect vulnerabilities may suffer, leading to increased false positives or undetected issues. For instance, if recent security advisories are not promptly incorporated, the LLM may miss emerging threats or base its recommendations on deprecated practices.

To mitigate this, future work is focusing on establishing a continuous integration pipeline for automatic updates from authoritative sources. Additionally, incorporating confidence scores or uncertainty estimation could help signal when the knowledge base might need refreshing, thus ensuring that the LLM's responses remain aligned with current standards. We could also explore if more up-to-date knowledge base content leads to fewer false positives.

B. CHALLENGES WITH SPECIALIZED OR CONDITIONAL TEMPLATES

Our evaluation revealed that templates incorporating advanced conditional logic or specialized configurations—such as nested conditions for resource creation, parameter-dependent resource properties, or non-standard formatting—pose significant challenges. For instance, when a template employs conditionals to instantiate resources only under certain environment settings, the LLM may either generate redundant vulnerability alerts or overlook risks tied to specific parameter combinations. These issues indicate that the LLM's current context extraction methods struggle to fully interpret dynamic behaviors encoded in the template structure.

To address these challenges, future work would need to focus on:

- **Enhanced Contextual Parsing:** Developing specialized parsers that extract and annotate conditional logic and parameter dependencies to provide a clearer context for the LLM.
- **Adaptive Prompt Engineering:** Crafting prompts that explicitly describe dynamic behaviors, such as conditional resource instantiation, to guide the LLM in understanding context-specific security implications.
- **Hybrid Analysis Approaches:** Integrating rule-based mechanisms to pre-screen conditional constructs, ensuring that only templates with clearly defined dynamic behaviors are passed to the LLM for detailed semantic analysis.

Experimental studies focusing on templates with known conditional complexities will be essential to quantify improvements and refine these strategies.

C. FUTURE DIRECTIONS FOR IMPROVEMENT

By conducting ablation studies to quantify the impact of an outdated knowledge base and by testing alternative strategies for handling complex templates, we can develop targeted solutions to enhance the overall robustness of our system. Incorporating a user feedback loop to fine-tune both the LLM and the retrieval process will further ensure that the approach adapts effectively to real-world variations and evolving security requirements. This feedback loop is crucial for iteratively refining prompts and model responses to reduce false positives and enhance overall accuracy based on real-world usage.

VII. FUTURE WORK

Building upon the current findings, several avenues exist for improving and extending our approach.

Initially, the incorporation of a feedback mechanism within the workflow would facilitate the refinement of both the suggestions generated by the LLMs and the retrieval process. By systematically documenting the input from developers—such as identifying a specific remediation as excessively stringent or recognizing a flagged issue as inconsequential—the system can adapt to actual usage patterns and progressively align more closely with established organizational standards.

Secondly, more detailed configuration options and policies might be integrated into the retrieval and analysis stages. By allowing organizations to delineate their own ''context modules'' (such as internal guidelines or sanctioned exceptions), the system could furnish recommendations aligned with internal standards, thereby not exclusively depending on external best practices.

Third, examining the incorporation of static checks prior to the invocation of the LLM could enhance accuracy whilst optimizing performance. A succinct rule-based prescreening process may address trivial cases and elevate more intricate scenarios to the LLM-based reasoning tier. This hybrid methodology could diminish inference durations and reduce computational overhead, thus rendering the solution more conducive to large-scale applications.

Finally, beyond AWS CloudFormation, generalizing the approach to other IaC frameworks (such as Terraform) and multi-cloud environments would broaden the solution's applicability. Carefully curating best-practice repositories for Azure, Google Cloud, or Kubernetes-based infrastructures would allow security teams to benefit from the same semantic reasoning capabilities across heterogeneous environments.

VIII. CONCLUSION

Our research elucidates that an LLM-driven solution, enhanced by a perpetually updated knowledge base, can markedly enhance the detection and remediation of security misconfigurations in AWS CloudFormation templates. Through the utilization of retrieval-based context, the LLM is capable of discerning subtle and context-dependent issues that are often overlooked by traditional static tools. This paradigmatic shift—from inflexible rule sets to a semantic, knowledge-driven analysis—affords developers and security teams more refined, actionable insights.

The evaluation indicated a significant increase in detection accuracy and contextual relevance. Although limitations persist in terms of knowledge maintenance, specialized environments, runtime costs, and occasional overly cautious remediation guidance, the overall improvements in security posture and developer assurance are apparent. The solution's versatile architecture and integration into CI/CD pipelines render it a promising advancement towards proactive, scalable, and context-aware cloud security practices. As the usage of IaC and cloud complexity continues to escalate, methodologies akin to this can assist organizations in confidently navigating the evolving landscape of infrastructure security.

REFERENCES

[1] D. Compton. (2024). *What Went Wrong With Unisuper and Google Cloud?*. [Online]. Available: https://danielcompton.net/google-cloud-unisuper

[2] cdklabs. (2024). *CDK-nag: Check CDK Applications for Best Practices Using a Combination of Available Rule Packs*. [Online]. Available: https://github.com/cdklabs/cdk-nag

[3] N. Saavedra and J. F. Ferreira, ''GLITCH: Automated polyglot security smell detection in infrastructure as code,'' 2022, *arXiv:2205.14371*.

[4] E. Malul, Y. Meidan, D. Mimran, Y. Elovici, and A. Shabtai, ''GenKubeSec: LLM-based kubernetes misconfiguration detection, localization, reasoning, and remediation,'' 2024, *arXiv:2405.19954*.

[5] X. Lian, Y. Chen, R. Cheng, J. Huang, P. Thakkar, M. Zhang, and T. Xu, ''Configuration validation with large language models,'' 2023, *arXiv:2310.09690*.

[6] F. Minna, F. Massacci, and K. Tuma, ''Analyzing and mitigating (with LLMs) the security misconfigurations of helm charts from artifact hub,'' 2024, *arXiv:2403.09537*.

[7] S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, and G. Stringhini. (2024). *LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet?): A Comprehensive Evaluation, Framework, and Benchmarks*. [Online]. Available: https://www.bu.edu/peaclab/files/2024/05/saad_ullah_llm_final.pdf

[8] (2024). *AWS Well-architected Framework*. [Online]. Available: https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html

[9] (2024). *CIS Benchmarks*. [Online]. Available: https://www.cisecurity.org/cis-benchmarks

[10] S. V. Datla and R. Stevens. (2024). *Introducing an Enhanced Version of the Aws Secrets Manager Transform: AWS::Secretsmanager-2024-09-16*. [Online]. Available: https://aws.amazon.com/blogs/security/introducing-an-enhanced-version-of-the-aws-secrets-manager-transform-awssecretsmanager-2024-09-16/

[11] (2024). *AWS CloudFormation User Guide*. [Online]. Available: https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/

DHEER TOPRANI received the dual bachelor's degree in computer science and commerce from the University of Virginia. He is currently pursuing the degree with the OMSCS Program, Georgia Institute of Technology. He is a System Development Engineer with Amazon Worldwide Returns and Recommerce, where he is focused on automating data and ETL pipelines at scale. He has also authored widely referenced AWS documentation and holds multiple AWS certifications in machine learning, data analytics, and cloud architecture. His research interests include LLM-based automation solutions, particularly in cloud security and infrastructure optimization.

VIJAY K. MADISETTI (Fellow, IEEE) received the Ph.D. degree in electrical engineering and computer sciences from the University of California at Berkeley. He is currently a Professor of cybersecurity and privacy (SCP) with Georgia Institute of Technology. He has authored several widely referenced textbooks on topics, including cloud computing, data analytics, blockchain, and microservices. Additionally, he has been honored with the Terman Medal by the American Society of Engineering Education (ASEE).